



Engineering Project Platform

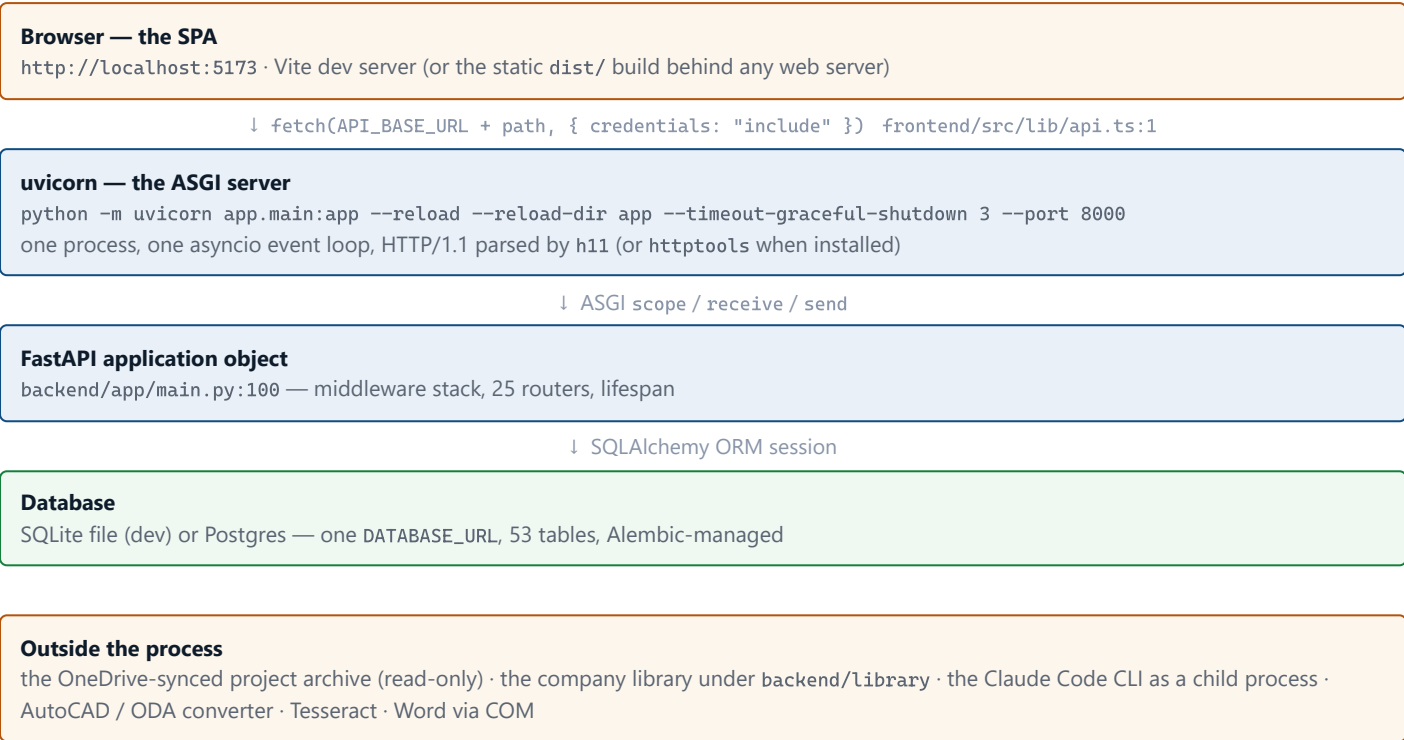
System Design — how a request travels through the stack

Repository C:\dev\ep-platform · branch master
Stack React 19 + Vite 8 → uvicorn + FastAPI 0.115 → SQLAlchemy 2.0 → SQLite / Postgres
Generated 22 September 2026, from the code as it stands — every hop cites file:line

1	Deployment topology	8	Trace E — file upload
2	The ASGI layer: bytes to Python	9	Trace F — export download
3	The universal request pipeline	10	Trace G — AI-assisted request
4	Trace A — POST /auth/login	11	Startup and shutdown
5	Trace B — GET /projects/{id}	12	The persistence layer
6	Trace C — PUT /projects/{id}	13	Component connection map
7	Trace D — long jobs (202 + polling)	14	Failure paths

1 Deployment topology

Two processes, started together by `start.bat`. There is no API gateway, no BFF and no server-side rendering: the browser talks straight to uvicorn across an origin boundary, which is why CORS and cookie settings matter as much as they do.



Why the origin boundary shapes the design

Consequence	Where it is handled
Cookies must be sent cross-origin	Client sets <code>credentials: "include"</code> ; server sets <code>allow_credentials=True</code> with an explicit origin list — <code>*</code> is illegal alongside credentials. <code>main.py:102</code>
The browser hides most response headers from JS	<code>expose_headers</code> names the six the SPA actually reads: <code>Content-Disposition</code> , <code>X-Package-Pages</code> , <code>X-Package-Warnings</code> , <code>X-Resource-Version</code> , <code>ETag</code> , <code>X-Request-ID</code> . <code>main.py:114</code>
Every non-simple request costs a preflight	<code>OPTIONS</code> is answered by <code>CORSMiddleware</code> and never reaches a router, a dependency or the database.
The session cookie is <code>samesite=lax</code>	Assumes frontend and API share a site in production. On different domains this must become <code>samesite=none; secure</code> . <code>auth.py:59</code>

2 The ASGI layer: how bytes become a Python call

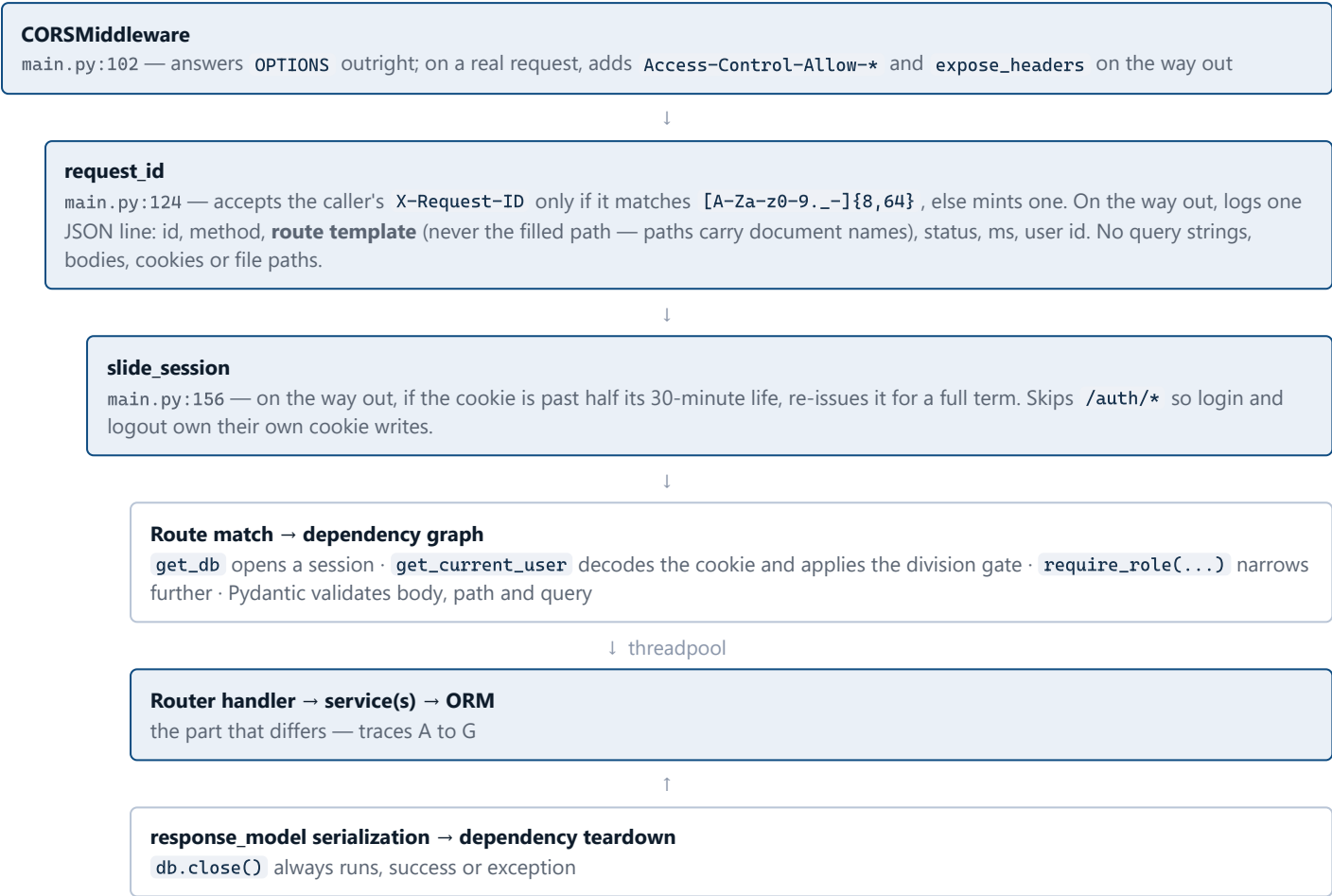
This is the part usually hand-waved. uvicorn is a single-threaded event loop; the FastAPI endpoints in this codebase are almost all plain `def`. The handoff between those two facts is the most important mechanical detail in the system.

#	Component	What happens to the request
1	OS socket uvicorn, port 8000	The event loop accepts the TCP connection. Nothing is parsed yet — only bytes have arrived.
2	HTTP protocol parser h11 / httptools	Request line, headers and body are parsed incrementally. A malformed request dies here with a 400 that no application code ever sees.
3	ASGI scope built uvicorn protocol layer	A dict: <code>type: "http", method, path, raw_headers, query_string, client</code> . The body is <i>not</i> in it — it is pulled later through the <code>receive</code> callable, which is how a large upload streams instead of buffering.
4	The app is called main.py:100	uvicorn awaits <code>app(scope, receive, send)</code> . <code>app</code> is the FastAPI instance, a subclass of Starlette. From here everything is a coroutine on that one event loop.
5	Middleware stack, outermost first main.py:102–190	Starlette wraps the app in each registered middleware. Registration order is <i>inside-out</i> : the last <code>@app.middleware</code> declared sits closest to the router. Effective order here: <code>CORS</code> → <code>request_id</code> → <code>slide_session</code> → router.
6	Routing starlette.routing	The path is matched against the route table built by the 25 <code>include_router</code> calls (main.py:191–242). First match wins, so prefix collisions matter: <code>/projects</code> is claimed by nine different routers. No match → 404 without touching the database.
7	Dependency resolution fastapi.dependencies	FastAPI walks the endpoint signature and builds the dependency graph <i>before</i> calling the handler: path params coerced, query params validated, the Pydantic body model constructed, <code>Depends(get_db)</code> and <code>Depends(get_current_user)</code> executed. A failure here is a 422 and the handler is never entered.
8	The threadpool handoff anyio worker thread	The critical hop. The endpoint is a sync <code>def</code> , so FastAPI does not await it — it runs it via <code>run_in_threadpool</code> on an anyio worker thread (40 by default). The event loop stays free to accept other requests. Everything the handler does — blocking SQLite I/O, reading a PDF off OneDrive, shelling out to AutoCAD — happens on that worker thread, never on the loop.
9	The handler runs app/routers/*.py	Router code calls services; services call the ORM. See traces A–G.
10	Serialization response_model=	The returned ORM object or dataclass is validated and filtered through the declared Pydantic model, then JSON-encoded. This is the field filter: a <code>User</code> row goes out as <code>UserOut</code> , so <code>hashed_password</code> cannot leak.
11	Unwind reverse middleware order	Dependency teardown (<code>get_db</code> closes the session in its <code>finally</code>) → <code>slide_session</code> may re-issue the cookie → <code>request_id</code> writes the log line and stamps the header → <code>CORS</code> adds its headers → <code>send</code> → bytes on the socket.

Sync vs async, counted. Across the 25 routers exactly **four** handlers are `async def`: `ifc_boq.py:477`, `ifc_boq.py:494`, their shared helper `ifc_boq.py:337`, and `submittal.py:854` — every one of them a handler that must `await` an `UploadFile`. All the rest are sync, and therefore threadpooled. Three consequences follow directly: the engine needs `check_same_thread: False` (`database.py:13`); shared in-process state needs real locks (`_fill_lock`, `design.py:77`; `_start_lock`, `verification.py:35`); and an async handler that wants to do blocking work must hand it back to a thread itself with `run_in_threadpool` (`ifc_boq.py:486`).

3 The universal request pipeline

Every one of the traces that follow runs through this same spine. Only the shaded band in the middle differs from endpoint to endpoint.



4 Trace A — POST /auth/login

The canonical trace: an unauthenticated request that ends with a session cookie. Every hop below is real code, not a sketch.

#	Component	What happens
1	SPA lib/api.ts	<code>request("/auth/login", { method: "POST", body: JSON.stringify({email, password}) })</code> with <code>credentials: "include"</code> and <code>Content-Type: application/json</code> . Because of the content type the browser first sends an <code>OPTIONS</code> preflight.
2	CORS preflight main.py:102	<code>OPTIONS /auth/login</code> is answered by the middleware alone: allowed origin, methods, headers, <code>Allow-Credentials: true</code> . No router, no dependency, no database.
3	uvicorn → ASGI	The real POST is parsed by h11, turned into a scope, and <code>app(scope, receive, send)</code> is awaited.
4	request_id main.py:124	Assigns <code>request.state.request_id</code> and starts the duration timer. It also peeks at the cookie to attribute the log line — on login there is none yet, so <code>user: null</code> .
5	slide_session main.py:156	Passes straight through on the way in. On the way out it returns early because the path starts with <code>/auth/</code> — login must own its own <code>Set-Cookie</code> .
6	Route match auth.py:16, 21	<code>APIRouter(prefix="/auth") + @router.post("/login")</code> → the <code>login</code> function. No auth dependency on this route — it is the one that creates the session.
7	Validation + session schemas.py / database.py:66	The JSON body is parsed into <code>LoginRequest</code> (a malformed body is 422 here). <code>Depends(get_db)</code> opens a <code>SessionLocal</code> .
8	Threadpool	<code>login</code> is a sync <code>def</code> → run on an anyio worker thread. <code>bcrypt</code> is deliberately slow; this is exactly why it must not run on the event loop.
9	Handler: lookup auth.py:22	<code>db.query(User).filter(User.email == payload.email.lower()).first()</code> → SQLAlchemy compiles <code>SELECT ... FROM users WHERE email = ?</code> , the pooled DBAPI connection executes it, the row is mapped to a <code>User</code> in the identity map.
10	Handler: gates auth.py:24–46	In order: unknown email → generic 401 (never "no such user"); <code>locked_until</code> in the future → 423 with minutes remaining; <code>is_active</code> false → 403; then <code>verify_password</code> (security.py:16, passlib/bcrypt).
11	Handler: failure path auth.py:41	A wrong password increments <code>failed_login_attempts</code> ; at <code>MAX_FAILED_LOGIN_ATTEMPTS</code> it sets <code>locked_until = now + LOCKOUT_MINUTES</code> and resets the counter. <code>db.commit()</code> runs before the 401 is raised — the lockout must survive the rejection.
12	Handler: success path auth.py:48–56	Counter cleared, <code>locked_until = None</code> , <code>last_login_at = now</code> , <code>db.commit()</code> , <code>db.refresh(user)</code> , then <code>activity.record(db, user, "auth.login", ...)</code> writes a row into <code>activity_events</code> — after the work succeeded, never before.
13	Token security.py:22	<code>create_access_token(subject=str(user.id), role=user.role.value)</code> → HS256 JWT with <code>sub</code> , <code>role</code> , <code>exp</code> , signed with <code>SECRET_KEY</code> .
14	Cookie auth.py:59	<code>response.set_cookie(httponly=True, samesite="lax", secure=COOKIE_SECURE, max_age=30*60, path="/")</code> . httpOnly is the point: the token is never readable by JavaScript, so an XSS bug cannot steal it.
15	Serialization response_model=UserOut	The <code>User</code> ORM object is filtered to <code>UserOut</code> . <code>hashed_password</code> , <code>failed_login_attempts</code> and <code>locked_until</code> are simply not in the model, so they cannot be sent.
16	Unwind	<code>get_db</code> closes the session → <code>slide_session</code> no-ops on <code>/auth/</code> → <code>request_id</code> logs <code>{"route": "/auth/login", "status": 200, "ms": 214, "user": null}</code> and sets <code>X-Request-ID</code> → CORS headers → response.
17	Back in the SPA context/AuthContext.tsx	The browser stores the cookie; the JSON user is put in <code>AuthContext</code> . <code>ProtectedRoute</code> and <code>RoleRoute</code> now let the app render. Every later request carries the cookie automatically.

Note the asymmetry. The token is created and signed on the server, and the SPA never sees it. The client's copy of "who am I" is only the `UserOut` JSON, which is advisory — the server re-decodes the cookie on every subsequent request. Client-side role checks are UI convenience; `deps.py` is the enforcement.

5 Trace B — authenticated read: GET /projects/{id}

The shape of most requests in the system. The difference from trace A is everything that happens *before* the handler.

#	Component	What happens
1	SPA	A simple <code>GET</code> with the cookie attached; no preflight, because no custom header or content type is set.
2	request_id <code>main.py:124</code>	Decodes the cookie <i>only</i> to attribute the log line, swallowing any error — it never rejects; that is the route's job.
3	slide_session <code>main.py:156</code>	Remembers to check the cookie's remaining life on the way out.
4	Route match <code>projects.py:69</code>	Nine routers share the <code>/projects</code> prefix. Starlette takes the first registered pattern that matches, in <code>include_router</code> order (<code>main.py:191–242</code>).
5	get_db <code>database.py:66</code>	Session opened. On SQLite, the pool hands out a connection and the <code>connect / checkout</code> listener re-runs <code>PRAGMA foreign_keys=ON</code> (<code>database.py:28</code>) — on <i>every</i> checkout, because migrations turn it off on the connection they borrow.
6	get_current_user <code>deps.py:12</code>	No cookie → 401. Bad or expired signature → 401 "Session expired or invalid". Valid → <code>db.get(User, int(sub))</code> , a primary-key load that may be served from the identity map.
7	The division gate <code>deps.py:30–37</code>	If the role maps to a division area, the request path must be <code>/auth/me*</code> or <code>{area}/projects*</code> . <code>/projects/7</code> is neither → 403 "This module is not available to your division". The design roles and admin pass.
8	Handler <code>projects.py:415</code>	<code>_get_project_or_404</code> (<code>projects.py:110</code>) does <code>db.get(Project, id)</code> → 404 if absent. Then <code>activity.record_open</code> (once per 30-minute window, because React StrictMode double-fires effects in dev) and <code>_ensure_project_folders</code> .
9	Lazy loads <code>models.py relationships</code>	Building the response touches <code>project.systems</code> , <code>project.documents</code> , <code>project.design</code> ... each an extra <code>SELECT</code> the first time it is read, inside the same session and transaction.
10	Serialization <code>schemas_project.py</code>	<code>response_model=ProjectOut</code> walks the loaded graph into JSON.
11	slide_session, on the way out <code>main.py:176</code>	If <code>exp - now ≤ 15 min</code> , a fresh 30-minute cookie is issued. So a long BOQ edit never expires mid-work; a session ends only after 30 minutes of silence.
12	Teardown	Session closed, log line written, headers added.

Where a read gets its data from — three different sources

The database

Project rows, BOQ lines, systems, submittals. The default.

The filesystem

The archive folder (`_project_folder`, `design.py:88`) and the company library. Read live, on the worker thread, which is why a stale OneDrive mount surfaces as 503 rather than a hang.

A cache table

`result_cache`, the datasheet index under `.cache/`, and the `ep_archive_folders` index — so a search is a query, not a directory walk.

6 Trace C — versioned write: PUT /projects/{id}

Two engineers can have the same project open. A write therefore has to say which version it was made against. This is the optimistic-concurrency path, and it runs on the BOQ, project info, design inputs and compliance statements alike.

#	Component	What happens
1	SPA lib/api.ts:78	The page kept the <code>X-Resource-Version</code> it received with the document. <code>ifMatch(version)</code> turns it into <code>If-Match: "4"</code> and sends it with the <code>PUT</code> . A request without the header is accepted — scripts and older clients still work.
2	Header binding projects.py:425	<code>if_match: str None = Header(default=None, alias="If-Match")</code> — FastAPI binds the raw header as a parameter.
3	Authorisation deps.py:39	<code>Depends(require_role(*CREATOR_ROLES))</code> — admin, design manager, design engineer. A viewer reaching this endpoint is 403 before any body is read.
4	Body validation schemas_project.py	<code>ProjectDetailsIn</code> . The EP number and document paths are deliberately not in the model — they tie the project to its archive folder and are not editable.
5	Load projects.py:434	<code>_get_project_or_404</code> .
6	The version check concurrency.py:43	<code>require_current(if_match, project.details_version, "The project information")</code> . <code>parse_if_match</code> accepts <code>"3"</code> , <code>W/"3"</code> or <code>3</code> ; <code>*</code> or absent means no check; anything else is a 400.
7	Conflict concurrency.py:46	If the version does not match → 409 with a structured body: <code>{code: "stale_write", message, your_version, current_version}</code> . The SPA's <code>ApiError.isStaleWrite</code> (api.ts:22) recognises it and renders <code>StaleWriteNotice</code> instead of a generic error.
8	Apply projects.py:436–440	<code>details_version += 1</code> , a before-snapshot is taken, fields applied, then <code>_propagate_details</code> pushes a brand or system change down into the BOQ lines.
9	Cascading version bump projects.py:450	If any BOQ item is dirty (<code>db.is_modified</code>), <code>boq_version += 1</code> as well — a BOQ table open in another tab must not save over what this write just changed.
10	Commit	One <code>db.commit()</code> flushes every change in the unit of work as a single transaction. Nothing was written before this point.
11	Audit activity.py:45	<code>activity.record(..., detail=changed)</code> stores a field-by-field <code>old → new</code> diff, truncated to 80 characters per field.
12	Response	Serialized as <code>ProjectOut</code> ; the new version travels back so the page can keep editing without a reload.

Two ways a version is bumped

Mechanism	Applies to
Explicit, in the router projects.py:436	<code>details_version</code> and <code>boq_version</code> are plain integer columns on <code>Project</code> , incremented by hand in the handler that changes them.
Automatic, by ORM event concurrency.py:64	<code>@event.listens_for(ProjectDesign, "before_update")</code> and the same for <code>ComplianceStatement</code> : any real modification bumps <code>version</code> , whoever caused it — a page save, a background AI job, or a recheck. A save made against the state before <i>any</i> of those is caught.

7 Trace D — long work: POST /projects/{id}/jobs/boq-reread

Re-reading a four-page scanned Design Sheet takes a minute or more. Run inside the request, the page could only spin, a refresh lost the answer, and there was no stopping it. The answer is a job row: the request returns **202** immediately and the work continues on a thread of its own.

#	Component	What happens
1	Handler, in the request jobs.py:66	<code>require_role(*CREATOR_ROLES)</code> → <code>_get_project_or_404</code> → refuse if the project has no Design Sheets (409) → <code>_refuse_duplicate</code> (jobs.py:57) returns 409 job_running with the existing job's id if one is already queued or running for this project and kind.
2	The closure jobs.py:77	A local <code>work(session, ctx)</code> is defined. It captures only ids — <code>project_id</code> , <code>user_id</code> — never the ORM objects from the request's session. Those belong to a session that is about to close.
3	Job row services/jobs.py:70	<code>jobs.start(...)</code> inserts a <code>background_jobs</code> row: <code>status="queued"</code> , <code>progress={done:0,total:0,message:"Waiting to start"}</code> , and commits it, so the row exists before the response leaves.
4	Thread launch services/jobs.py:81	A <code>threading.Thread</code> is started running <code>run()</code> , which opens its own <code>SessionLocal()</code> . This is the point people get wrong: the request's session and the job's session are two different sessions on two different connections, and the request's is closed the moment the response is sent.
5	Response jobs.py:88	202 Accepted with the <code>JobOut</code> body. The request is over; the work has barely begun.
6	The worker services/jobs.py:83	<code>status="running"</code> , <code>started_at</code> set and committed, then <code>work(session, JobContext(job_id))</code> runs — here <code>boq_candidates.build</code> , which reads the sheets, calls the AI reader and writes a candidate BOQ.
7	Progress services/jobs.py:38	<code>ctx.progress(done, total, message, **extra)</code> opens yet another short-lived session , writes the progress JSON and commits, then closes it. A separate session on purpose: progress must become visible to the polling request even though the job's own long transaction is still open.
8	Cancellation services/jobs.py:38, 53	The same read that writes progress also reads <code>cancel_requested</code> ; if set, it raises <code>Cancelled</code> . So a job stops <i>between</i> steps — at the next page, never mid-write.
9	Completion services/jobs.py:87	<code>status="succeeded"</code> , <code>result</code> (the candidate id and a summary), <code>finished_at</code> . On an exception the traceback is stored in <code>error</code> and the status is <code>failed</code> .
10	Polling lib/useJob.ts → jobs.py:130	The page polls <code>GET /jobs/{id}</code> — an ordinary authenticated read of one row — and renders <code>JobProgress</code> . A refresh loses nothing: the job is found again by id.
11	Stop jobs.py:139	<code>POST /jobs/{id}/cancel</code> only sets the flag. The worker notices it at its next checkpoint.

A job that was running when the server stopped cannot be resumed. On the next start, `jobs.fail_interrupted` (services/jobs.py:124, called from the lifespan at main.py:70) marks every still-**running** row as failed with that reason — so no page waits forever on work that died with the process.

Endpoints that follow this pattern

`jobs/boq-reread` and `jobs/documents-intake` (jobs.py), `jobs/ai-verify` (verification.py), and `ifc-drawings/jobs` (ifc_boq.py:494), which additionally reports a percentage, a stage name and an estimated seconds-remaining through the same `progress` channel.

8 Trace E — upload: POST /projects/{id}/ifc-drawings

The only place in the codebase where the `async/sync` distinction is visible in the source, and the clearest illustration of why the threadpool exists.

#	Component	What happens
1	SPA <code>components/ifc/api.ts</code>	A <code>FormData</code> POST. The browser sets <code>multipart/form-data</code> with its own boundary — the client must <i>not</i> set <code>Content-Type</code> itself.
2	ASGI body streaming	The body is not in the scope. Starlette pulls it in chunks through <code>receive</code> , spilling to a temporary file past a size threshold, so a 60 MB DWG does not sit in memory as one string.
3	The async handler <code>ifc_boq.py:477</code>	<code>async def upload_drawing(...)</code> — declared <i>async because</i> it must <code>await</code> the upload. It runs on the event loop , not on a worker thread.
4	Await the bytes <code>ifc_boq.py:337</code>	<code>await _received(file)</code> → filename, extension and the bytes. Awaiting yields control, so other requests keep being served while the upload arrives.
5	Back to a thread <code>ifc_boq.py:486</code>	<code>await run_in_threadpool(lambda: _read_drawing(...))</code> . Everything blocking — converting DWG to DXF through AutoCAD's core console or the ODA converter, parsing with <code>ezdxf</code> , matching blocks against <code>ifc_symbols</code> , writing rows — happens on a worker thread. Without this one call the event loop would freeze for the whole conversion and the entire server would stop answering.
6	Failure <code>ifc_boq.py:488</code>	A <code>ReadError</code> becomes 422 with the reason. A missing converter is refused up front with an instruction to install AutoCAD or the ODA converter, or to upload a DXF instead (<code>ifc_boq.py:511</code>).
7	Or, as a job <code>ifc_boq.py:494</code>	The tab actually uses the sibling endpoint, which does steps 3–4 in the request and then hands the parsed bytes to trace D, so the page can show a stage and an ETA rather than a spinner.

Where uploaded bytes land

Never in the archive. `UPLOADS_ROOT` (default `backend/uploads`, or `<DATA_ROOT>/uploads` when a data root is set, `config.py`) gets a folder per project; `_save_upload` (`projects.py:1152`) writes the file and the path is stored on the row. The platform **reads** the OneDrive archive and **writes** only to its own uploads folder — a deliberate one-way boundary.

9 Trace F — export: a generated file comes back

Excel exports, the battery calculation PDF, a built submittal package. Same pipeline, different exit.

#	Component	What happens
1	Browser	Either a <code>fetch</code> whose blob is saved, or a plain link to <code>apiUrl(path)</code> (<code>api.ts:4</code>) for something the browser can display, such as a datasheet PDF.
2	Handler <code>drawings.py:114</code> , <code>boq_review.py</code>	Normal auth and load, then a service builds the file in memory : <code>boq_export</code> , <code>battery_export</code> , <code>schedule_export</code> , <code>submittal_export</code> — <code>openpyxl</code> into a <code>BytesIO</code> , or <code>PyMuPDF</code> for a PDF.
3	Response object	<code>StreamingResponse(buf, media_type=..., headers={"Content-Disposition": f'attachment; filename="..."'})</code> , or <code>FileResponse</code> when the bytes are already a file on disk (<code>design_rules.py:328</code> , <code>projects.py:1321</code> , both <code>content_disposition_type="inline"</code> so the browser displays rather than downloads).
4	CORS, the catch <code>main.py:114</code>	Cross-origin, JavaScript cannot read <code>Content-Disposition</code> unless it is in <code>expose_headers</code> — nor <code>X-Package-Pages</code> / <code>X-Package-Warnings</code> , which is exactly why the submittal page once reported "? pages assembled".
5	Logging <code>main.py:141</code>	The log line records the route template , never the filled path, so a document name or a client's folder never reaches the log.

Word and PDF. The warranty letter is converted by driving Word over COM (`pywin32`, Windows only) so the page carries the real letterhead; if the import is unavailable the code falls back to a drawn page. That subprocess call, like every other blocking call, is safe only because the handler is on a worker thread.

10 Trace G — an AI-assisted request

Off by default (`AI_ENABLED=false`). When on, a model is asked only about things the deterministic path could not settle, and its answer is a *proposal* an engineer accepts — never a value the system writes by itself.

1 Router

`verification.py`, `extraction.py` — usually starting a job (trace D), because a read takes minutes

↓

2 Eligibility

`ai/evaluation.py`, `ai/project_policy.py` — is this task allowed on this server and this project? `AI_DISABLED_TASKS` is the deployment kill switch, per task, without a code change

↓

3 Budget

`ai/budget.py` — calls per document, calls per project per day, elapsed seconds, estimated cost. Exceeded → the task stops and says so

↓

4 Cache

`ai/cache.py` → `result_cache`. The key includes `PROMPT_VERSION` (`provider.py:33`), so re-wording a prompt invalidates old answers. A document with the same content is never read twice

↓

5 Guard — document text is data

`ai/guard.py:49` `neutralise()` makes every tag-shaped sequence inert so a document cannot close the fence it is wrapped in; `guard.py:66` `scan_parts()` flags instruction-like wording. A flagged call still runs — the words may be innocent — but its proposal can never be `validated`: it goes to an engineer with the flag shown

↓

6 AiRequest

`provider.py:48` — task, system prompt, fenced text and PNG parts, a JSON schema, token ceiling, tier (`small` / `standard`), idempotency key

↓

7 `get_provider().complete()`

`provider.py:581` — the one place a model is called. Nothing else imports a vendor SDK

ClaudeCodeProvider (:450, default) spawns the Claude Code CLI as a child process on the subscription signed in on the server — no API key · **ClaudeProvider** / **OpenAiProvider** call the vendor APIs with a key · **NullProvider** (:121) answers "insufficient evidence" to everything and is what runs when AI is off — the deterministic pipeline must be complete with it · **RecordingProvider** is what the tests use

↓

8 Validation

`ai/proposals.py` — refuses a target, region or value the task does not allow; `suspicious_value` catches links, markup or instruction wording that leaked into an answer

↓

9 Persistence

`ai_proposals` (what was suggested) · `ai_usage` (tokens and estimated cost, per call) · `ai_verifications` (a run and the items it touched) · `document_readings` (a reading kept for good) · `result_cache`

↓

10 The engineer decides

`AiVerificationPanel.tsx`, `ExtractionReview.tsx` — accept, reject, or undo a run that changed something

Reading documents is AI-only. The DRF and the Design Sheets are read by the model (`ai/sheet_reader.py`, `ai/verification.py`) — there is no OCR witness and no fallback; when the model cannot read something it says "*Not read: <why>*". Tesseract remains, but only for document intake, document control and the submittal scanner. The deterministic readers (`services/drf_extractor.py`, `services/design_sheet_extractor.py`) no longer read documents; their parsers and page rendering are reused by the AI read, and they are kept with their tests until the AI-only read has run on enough projects.

11 Startup and shutdown

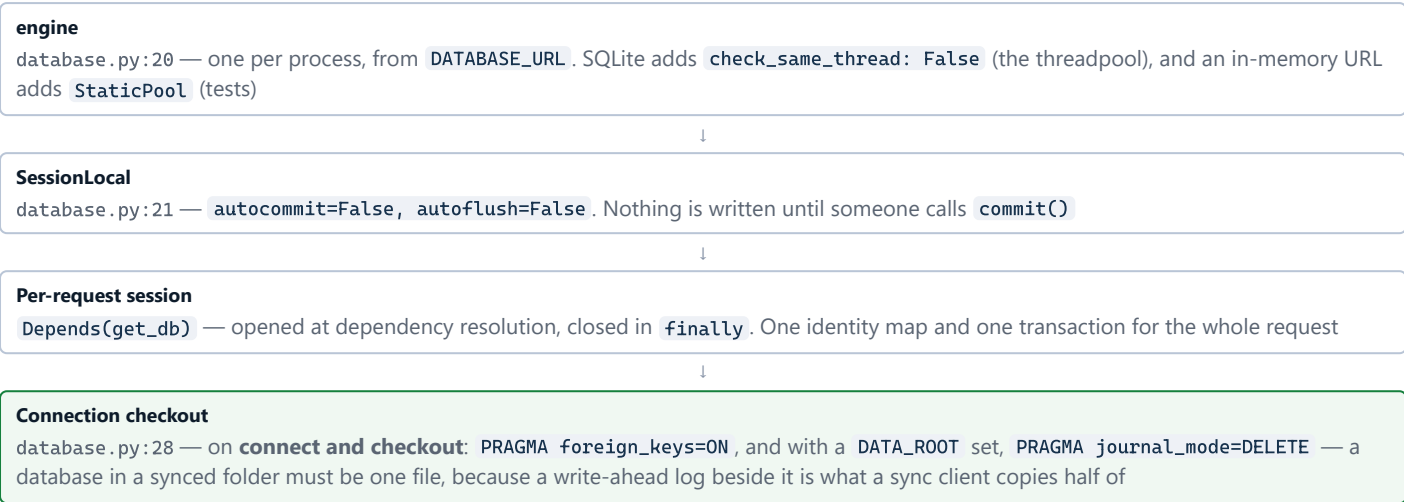
Before the first request can be served, the lifespan context manager (`main.py:41–97`) runs in strict order. Anything slow enough to make a new machine wait is pushed onto a background thread instead.

#	Step	Why it is there, and why in this order
1	adopt_local_database <code>main.py:45</code> → <code>database.py:44</code>	First start after <code>DATA_ROOT</code> is pointed at a synced folder: an existing <code>backend/ep_platform.db</code> is copied in, so nothing made on this PC is left behind.
2	upgrade_to_head <code>main.py:48</code> → <code>migrations.py</code>	<code>alembic upgrade head</code> in-process, against the same <code>DATABASE_URL</code> . Pull a change, restart, done. This replaced <code>Base.metadata.create_all</code> , which created missing tables but never altered existing ones — a new column silently did not appear and the app failed at query time.
3	Seeds <code>main.py:51–70</code>	In one session: default admin, design rules, equipment currents, datasheet links, suppliers, IFC device types. Then <code>import_library</code> / <code>save_library</code> round-trips the IFC symbol library with the file on the company shelf, so database and file always agree. Every seed is idempotent.
4	fail_interrupted <code>main.py:74</code>	Jobs left <code>running</code> by a killed process are marked failed with that reason. See trace D.
5	Warm the libraries <code>main.py:80</code>	One daemon thread per datasheet library. Reading every datasheet over a synced drive takes a while; doing it now beats doing it on the first lookup.
6	Knowledge import <code>main.py:85</code>	On a machine that has never imported it, the compliance knowledge base is read from <code>data base/</code> in the background, so Auto-fill works without anyone configuring anything.
7	Archive index <code>main.py:93</code>	A refresh thread walks the archive's EP folders into <code>ep_archive_roots</code> / <code>ep_archive_folders</code> . A first scan takes minutes over OneDrive and nothing waits for it; afterwards, typing an EP number is a database query rather than a directory walk. Refreshed every <code>ARCHIVE_INDEX_REFRESH_MINUTES</code> (30).
8	yield <code>main.py:95</code>	uvicorn begins accepting requests.
9	stop_refresh <code>main.py:97</code>	On shutdown the index thread is stopped. <code>--timeout-graceful-shutdown 3</code> means a reload waits at most three seconds for open connections instead of hanging on them with the old code still serving.

Config is resolved once, at import. `get_settings()` is `@lru_cache d` (`config.py`), and its `model_validator`s run then: `~` and `%VAR%` expanded, the OneDrive archive located by name (`find_synced_folder`), Tesseract found on the PATH or where the installer puts it, and `DATA_ROOT` rewritten into `DATABASE_URL`, `UPLOADS_ROOT`, `CACHE_ROOT` and `BACKUPS_ROOT`. A wrong value in `.env` therefore shows up at start, not mid-request — with one exception: an AI budget set too low is only discovered when a call is actually made.

12 The persistence layer

Session lifecycle



Who else opens a session

Opener	Lifetime
The request (get_db)	One per HTTP request.
A job worker (services/jobs.py:82)	One per job, on its own thread, for the whole run.
A progress tick (services/jobs.py:39)	Opened, written, committed and closed per tick — so progress is visible while the job's own transaction is still open.
The lifespan (main.py:49)	One, for the seeds, closed before yield .

Schema: 53 tables, two hubs

34 foreign keys point at users.id ; 25 point at projects.id . Everything else clusters around those two.

Cluster	Tables
Identity	users , activity_events
Division silos	estimation_projects , division_projects — separate tables, not rows in projects . They carry reference, title, client and creator only; none of the workspace machinery hangs off them
Project core	projects (unique ep_number ; status draft → active → archived), project_systems , project_boq_items , project_designs , project_design_sheets , project_floor_schedule , project_frc_cables , project_amplifier_design , project_proposed_materials , battery_panel_results
Versions	boq_snapshots , project_boq_revisions , boq_candidates
Documents	project_documents , document_dependencies , document_readings , project_submittals → project_submittal_events
Extraction & AI	extraction_runs → extraction_issues , ai_verifications , ai_proposals , ai_usage , result_cache
Compliance	compliance_statements , compliance_learned_answers , compliance_audit
Knowledge	knowledge_imports (8 FKs into it) → files, sources, requirements, mappings, mapping_reviews, responses, response_sources, models, equivalences, issues
IFC	ifc_device_types → ifc_symbols , ifc_block_aliases , project_ifc_drawings
Reference / index	design_rules , equipment_currents , brand_suppliers , part_datasheet_links , ep_archive_roots → ep_archive_folders , background_jobs

Changing the schema. Edit models.py → alembic revision --autogenerate -m "what changed" → read the generated file (autogenerate misses things, and SQLite needs batch_alter_table , which env.py turns on) → restart. The test suite builds its database through the migrations, and tests/test_migrations.py fails when models and migrations disagree — a model change without a migration is caught there, not in the running app.

13 Component connection map

Who calls whom. The rule the codebase keeps: routers do HTTP, services do work, only services touch the filesystem and only `provider.py` touches a model.

Layer	Modules	Talks to
SPA pages	29 in <code>src/pages/</code> , plus <code>components/ifc/</code> and <code>components/drawings/</code>	<code>lib/api.ts</code> only. Domain helpers (<code>boq.ts</code> , <code>quantity.ts</code> , <code>ve.ts</code>) are pure functions, no I/O
Middleware	<code>CORS</code> , <code>request_id</code> , <code>slide_session</code>	Every request; nothing application-specific. <code>slide_session</code> reads <code>core/security</code>
Dependencies	<code>deps.get_current_user</code> , <code>deps.require_role</code> , <code>database.get_db</code>	<code>core/security</code> , <code>models.User</code> . Every protected route
Routers (25)	<code>app/routers/</code>	Services, schemas, <code>concurrency</code> , <code>activity</code> , <code>jobs</code> . Routers import each other's helpers freely — <code>_get_project_or_404</code> and <code>CREATOR_ROLES</code> come from <code>projects.py</code> , <code>JobOut</code> and <code>_refuse_duplicate</code> from <code>jobs.py</code>
BOQ services	<code>boq_candidates</code> , <code>boq_export</code> , <code>boq_provenance</code> , <code>boq_revisions</code> , <code>schedule_materials</code> , <code>schedule_export</code>	<code>boq_review.py</code> , <code>jobs.py</code> → <code>project_boq_items</code> , <code>boq_snapshots</code> , <code>boq_candidates</code>
Calculation services	<code>battery_*</code> , <code>amplifier_*</code> , <code>power_*</code> , <code>ve_calculation</code> , <code>ve_workbook_reader</code> , <code>calc_integrity</code>	<code>design.py</code> , <code>amplifier.py</code> → <code>design_rules</code> , <code>equipment_currents</code> , <code>project_designs</code> . Pure functions over inputs; the rule version used is stored with the result
Document services	<code>document_intake</code> , <code>document_control</code> , <code>document_sync</code> , <code>drawing_log</code> , <code>required_drawings</code> , <code>transmittals</code> , <code>submittal_*</code> , <code>word_text</code>	<code>documents.py</code> , <code>drawings.py</code> , <code>submittal.py</code> → the archive (read), <code>UPLOADS_ROOT</code> (write), Tesseract, Word/COM
Library services	<code>datasheet_library</code> , <code>datasheet_links</code> , <code>datasheet_currents</code> , <code>part_catalog</code> , <code>company_library</code> , <code>suppliers</code> , <code>spec_finder</code>	<code>materials.py</code> , <code>design.py</code> → <code>backend/library/</code> , the index under <code>.cache/</code> . A listing is trusted for <code>LIBRARY_RESCAN_SECONDS</code> (60) — looking up fifty parts used to re-walk the folder fifty times
Archive services	<code>ep_directory</code> , <code>ep_resolver</code> , <code>project_directory</code> , <code>project_folders</code> , <code>project_deletion</code>	<code>archive.py</code> , <code>projects.py</code> , <code>data_location.py</code> → the synced archive (read-only) and the index tables
AI	<code>app/ai/</code> — <code>guard</code> , <code>budget</code> , <code>evaluation</code> , <code>cache</code> , <code>provider</code> , <code>proposals</code> , <code>sheet_reader</code> , <code>verification</code> , <code>submittal_reader</code> , <code>metrics</code>	<code>extraction.py</code> , <code>verification.py</code> , <code>compliance/assist.py</code> . Only <code>provider.py</code> imports a vendor SDK or spawns the CLI
IFC	<code>app/ifc/</code> — <code>dxf/convert</code> , <code>dxf/extract</code> , <code>dxf/matcher</code> , <code>resolve</code> , <code>library_file</code> , <code>comparison</code> , <code>export</code>	<code>ifc_boq.py</code> → AutoCAD core console / ODA converter, <code>ezdxf</code> , the symbol tables
Compliance / knowledge	<code>app/compliance/</code> , <code>app/knowledge/</code>	<code>compliance.py</code> , <code>knowledge.py</code> → the knowledge tables, <code>Compliance_Response_Database.xlsx</code> , <code>openpyxl</code> / <code>xlrd</code>
Infrastructure	<code>jobs</code> , <code>concurrency</code> , <code>activity</code> , <code>readiness</code>	Used by almost every router. <code>concurrency</code> also registers ORM events, so it acts without being called

14 Failure paths

Which layer produces which status, and what the SPA does with it.

Status	Raised by	Meaning, and the client's response
400	<code>concurrency.parse_if_match</code> <code>concurrency.py:31</code>	An <code>If-Match</code> that is not a version number.
401	<code>get_current_user</code> <code>deps.py:15, 20, 25</code>	No cookie, an invalid or expired token, or a user who no longer exists or is disabled. The SPA drops to the login route.
403	<code>get_current_user</code> (division gate) <code>deps.py:35</code> ; <code>require_role</code> <code>deps.py:42</code> ; <code>login</code> <code>auth.py:40</code>	Authenticated but not entitled — wrong division, wrong role, or a disabled account at login. Routed to <code>AccessDeniedPage</code> .
404	Starlette router, or <code>_get_project_or_404</code> <code>projects.py:110</code>	No such route, or no such row.
409	<code>require_current</code> <code>concurrency.py:46</code>	<code>stale_write</code> — someone saved after this page loaded. Structured body with both versions; <code>ApiError.isStaleWrite</code> renders <code>StaleWriteNotice</code> rather than a generic error.
409	<code>_refuse_duplicate</code> <code>jobs.py:57</code>	<code>job_running</code> — the same work is already under way; the body names the job id so the page can attach to it.
409	<code>_project_folder</code> <code>design.py:89</code>	The project has no archive folder recorded.
422	FastAPI validation, before the handler	A bad body, path or query value. <code>errorFrom</code> (<code>api.ts:40</code>) names the first field that failed rather than showing a raw error list.
422	<code>ReadError</code> <code>ifc_boq.py:488</code>	A drawing that could not be read, or no DWG converter on this PC — with the fix in the message.
423	<code>login</code> <code>auth.py:35</code>	Locked after <code>MAX_FAILED_LOGIN_ATTEMPTS</code> ; the message says how many minutes remain.
503	<code>_project_folder</code> <code>design.py:92</code>	The archive folder is not reachable — OneDrive unmounted or still syncing. Distinct from 409 on purpose: one is a data problem, the other is an environment problem.
500	Anything unhandled	The <code>request_id</code> middleware still logs the line (its <code>finally</code> defaults the status to 500) and still returns <code>X-Request-ID</code> , so a user-reported failure can be found in the log.

What holds the whole thing together

One place per concern

Auth in `deps.py`. Versioning in `concurrency.py`. Long work in `services/jobs.py`. Model calls in `provider.py`. Config in `config.py`. Each is small enough to read in one sitting, and nothing routes around them.

Fail visibly, not silently

Migrations run at startup; interrupted jobs are failed on boot; a stale write is refused rather than merged; an AI answer is a proposal, not a write; a document that cannot be read says *"Not read: why"*.

Read wide, write narrow

The archive and the company library are read; only the platform's own database and `UPLOADS_ROOT` are written. That one-way boundary is why a sync conflict can never destroy project data.